

DefectLens — technical report

AI ARENA 2026 — the 8-hour AI/ML hackathon of DRESTEIN'26, the 17th National Level Intercollegiate Technical and Management Fest of Saveetha Engineering College (SEC), Chennai, presented by its Department of Artificial Intelligence and Machine Learning · **Track:** DefectLens, industrial visual inspection · **Team 18, Sri Manakula Vinayagar Engineering College (SMVEC), Puducherry** · Challenge and data: sanjai-umashankar/AI-Arena-AIML-Hackathon-2026

In short: a pretrained ResNet18 fine-tuned on *disturbed* copies of the training photos, measured by a 29-condition stress test rather than by the validation set alone, and delivered as an inspection station that answers PASS / REJECT / REVIEW with a heat-map. The naive model scores 0.985 macro-F1 on validation and 0.292 under one stress condition. One of our models scores 1.000 on validation and 0.990 ± 0.002 averaged over the 29 conditions. The submitted three-model ensemble scores 0.996, and 0.980 at its worst.

1. Problem understanding

Six-way image classification — `normal`, `scratch`, `dent`, `contamination`, `misalignment`, `missing_component` — scored by **macro-F1 on a private test set**, with a confidence per answer.

The organisers state that the private test "deliberately changes lighting, orientation, backgrounds and defect presentation" and warn that "a high validation score alone does not guarantee a high final score". We therefore treated the task as a **robustness problem**. Getting the given photos right is not enough: the answer has to survive changes of conditions the model has never seen. Everything below follows from that.

Two consequences shaped the work:

- **Validation cannot be the main yardstick.** It shares the training photos' conditions (with one exception, §2), so it measures what we have, not what we will be marked on.
- **Confidence matters.** The judging awards 15 marks for robustness and unknown handling, and a real inspection line needs to know when *not* to trust the machine.

2. Dataset analysis

	train	validation
images	750 (125 per class)	200 (33–34 per class)
size	128 × 128 RGB PNG	same

The images are synthetic: a grey plate carrying three dark component dots, tilted, on a pale background. Each defect is drawn in one consistent way (`docs/sheet_train.png`):

Class	Appearance in training
normal	three dark dots
scratch	thin short dark lines
dent	extra circles with a dark outline and lighter fill
contamination	small brown / orange / green specks — the only chromatic class
misalignment	two long lines crossing in an X
missing_component	one dot rendered white

Shortcuts that work in training and should fail in the private test (the "hidden layers" of the dataset):

1. **Glare squares exist only in validation.** Every validation class carries pale grey/white rectangles; no training image does (`docs/sheet_validation.png`). This is one of the test's shifts shown to us in advance. It targets `missing_component` , whose defect is itself a white blob.
2. **Colour identifies contamination.** A global colour cast (warm lighting) makes a colour-keyed model call everything contamination.
3. **Orientation** is limited to roughly $\pm 40^\circ$ in training.
4. **Backgrounds** are always flat pale colours.
5. **Label leakage through the IDs.** The number in every file name modulo 6 gives the class exactly, with a different mapping in train and validation (verified on all 950 files). The private IDs almost certainly follow the same scheme. Using it would be inferring hidden labels, which the rules forbid. **No code in this project reads the ID number;** `data.py` documents why.

We also checked the package for hidden content — zip comments, trailing data, extra PNG chunks, unlisted files, repository history — and found none.

Class balance: perfectly balanced, so accuracy and macro-F1 nearly coincide here. Macro-F1 still protects us if the private test is imbalanced.

3. Preprocessing

Deliberately minimal, so nothing needs to be re-created identically at test time:

- `standardise()` (`data.py`): any input is converted to RGB, padded to a square with its own edge colour if needed (squashing would distort the part), and resized to 128×128 . A no-op on the organisers' images, pixel for pixel (verified). It exists so the prototype and the private test cannot crash on an unexpected size.
- **Inside the model** (`model.py`): bilinear upscaling to 224×224 and ImageNet mean/std normalisation, because the backbone was pretrained on that.

Robustness does not come from preprocessing. It comes from the training-time shift simulator (§5).

4. Model selection

ResNet18 pretrained on ImageNet, final layer replaced by a 6-way linear layer, all layers fine-tuned.

- *Pretrained rather than from scratch*: 750 images is little data. The early layers already encode edges, lines, circles and textures, which are the primitives of every defect here.
- *ResNet18 rather than something larger*: 11M parameters, about 2 minutes per full training run on a laptop GPU (Apple M3 Pro, MPS). That made it affordable to train **nine models** for the controlled comparisons below, which a larger network would not have allowed in the time. Capacity was not the limit: validation reaches 1.000.
- *Rejected — a hand-written rules engine* (count dark dots, detect long lines, measure saturation). Transparent, but brittle to exactly the "changed defect presentation" the organisers promise.

5. Training methodology

Optimisation. AdamW (lr 5e-4, weight decay 1e-4), one-cycle schedule (10% warm-up, cosine decay), batch 32, 30 epochs, cross-entropy with **label smoothing 0.1**. We keep the **last** epoch, not the best-validating one, so the validation score stays an honest estimate.

The shift simulator (`shifts.py`) disturbs every training image on the fly, one family per shift the organisers named. Each fires with some probability, so clean and disturbed copies mix.

Family	Disturbances
orientation	any angle 0–360°, mirror, zoom 0.6–1.2×, shift ±10 px (corners filled with the background colour so rotation adds no artefact)
lighting	brightness, contrast, gamma, RGB colour cast, one-sided illumination, hue rotation, greyscale
background	part cut out (edge-connected region matching the frame colour; the part's outline stops it) and placed on dark / light / coloured / gradient / textured backgrounds; skipped when the cut-out is implausible (10 of 750 images)
occlusion	1–3 pale translucent rectangles (glare, stickers)
camera	Gaussian blur, sharpening, Gaussian noise, salt-and-pepper, JPEG compression

`docs/shift_examples.png` shows each family on one part per class. We rendered this sheet before training anything, because a faulty disturbance silently degrades every result downstream.

Final models: the same recipe on train + validation together (950 images), three times with seeds 42, 1 and 2, trained once every decision had been made on separate runs.

6. Validation methodology and results

6.1 Why validation alone was not enough

The naive model — identical, but no disturbances — scores **0.985** on validation. After two epochs *any* model scores ~0.97. Validation cannot distinguish a robust model from a fragile one.

6.2 The stress test (`evaluate.py`)

The 200 validation images re-drawn under **29 fixed-seed conditions**, each scored separately. Every condition is labelled by its relation to the training recipe, because a condition the model practised proves less than a

surprise:

- **inside range** — a disturbance the simulator makes, at a strength it uses
- **beyond range** — a disturbance it makes, stronger than training ever did
- **never simulated** — tilted camera (perspective), motion blur, shadow, low resolution

`docs/stress_conditions.png` shows all 29.

6.3 Results (macro-F1, single view)

Condition	kind	naive	v1	v2	v3 (final recipe)
validation as given	—	0.985	1.000	1.000	1.000
turned 90°	inside	0.944	1.000	1.000	1.000
any angle	inside	0.914	0.893	0.975	0.970
black & white	inside	0.774	1.000	1.000	1.000
far away (0.65×)	inside	0.934	0.710	1.000	0.990
salt & pepper	inside	0.292	0.586	0.047	0.985
sharpened	inside	0.754	0.606	0.523	0.995
dim light	beyond	0.951	1.000	1.000	0.995
warm light	beyond	0.926	1.000	1.000	1.000
heavy glare	beyond	0.859	1.000	0.995	0.995
blur (σ 2.5)	beyond	0.666	0.956	0.964	0.956
grain (σ 20)	beyond	0.347	1.000	1.000	0.995
squeezed JPEG (q 15)	beyond	0.798	0.990	0.995	0.990
everything at once	beyond	0.413	1.000	1.000	0.990
very far (0.5×)	beyond	0.523	0.096	0.960	0.783
tilted camera	never	0.980	1.000	1.000	1.000
motion blur	never	0.970	0.995	0.995	1.000
shadow	never	0.985	1.000	1.000	1.000
low resolution	never	0.914	0.990	0.985	0.990
mean, all 29		0.849	0.925	0.945	0.987
mean, beyond range		0.797	0.926	0.992	0.975
mean, never simulated		0.962	0.996	0.995	0.998

Full table for all nine models: `outputs/stress_table_all_models.md` . With eight-view test-time averaging, v3 reaches **0.993** over all 6,000 stressed images (0.987 with one view).

6.3a Run-to-run variation and the submitted ensemble

The v3 recipe was trained with three seeds (42, 1, 2) to measure how much of each number is luck (`outputs/stress_table_seeds.md` , single view):

	seed 42	seed 1	seed 2	mean ± std
mean over 29 conditions	0.987	0.992	0.992	0.990 ± 0.002
very far (0.5×)	0.783	0.901	0.926	0.870 ± 0.062
blur	0.956	0.980	0.970	0.969 ± 0.010

The average is stable. The half-size case is not, and our first run happened to be the weakest seed. The **submitted pipeline** averages all three seeds, each seeing every photo 16 ways (8 turns/mirrors × zoom 1.0 and 1.3). Zoomed views were added after the stress test identified half-size parts as the weak case, so that particular gain is partly fitted to our own stress set:

Pipeline (v3 recipe)	mean over 29	worst condition	very far
one model, one view	0.987	0.783	0.783
one model, 8 views	0.992	0.870	0.870
one model, 16 views (with zoom)	0.996	0.965	0.990
three seeds, 16 views — submitted	0.996	0.980	0.990

(`outputs/stress_table_tta_zoom1.md` , `..._zoom13.md` , `..._ensemble.md` .)

6.4 Ablation — what each family is worth

Each family switched off in turn (v1 recipe), everything else identical:

Family off	Mean under stress	Clearest damage
none (v1)	0.925	—
orientation	0.863	turned 90°: 1.000 → 0.563
occlusion	0.837	validation itself: 1.000 → 0.948 ; heavy glare 0.779
camera	0.881	squeezed JPEG 0.702, grain 0.807
background	0.907	modest, spread across conditions
lighting	0.936	colours turned 0.932, black & white 0.965 — <i>but</i> far away improves (see §7)

The occlusion row is the evidence that pale-rectangle augmentation is what defeats the validation set's glare trap.

6.5 Calibration

On the pooled stressed images, the best temperature is **T = 0.50**: the model is *under*-confident, because label smoothing caps its probabilities near 0.9 while it is right 99% of the time. Expected calibration error falls from 0.088 to 0.001 at T = 0.50. **We deliberately do not apply T < 1** (`predict.py` , `temperature_for`). The stress test is our own model of the private shifts, and sharpening confidence on the strength of our own model is the risky direction: an over-confident wrong answer lets a defective part pass. Confidence is only ever allowed to become more cautious.

6.6 What to expect on the hidden test

Our numbers should be read as an **upper bound**, for three reasons:

1. **Validation is easy.** Every model we trained scores 0.95–1.00 on it, the naive one 0.985. A perfect validation score is a sanity check, not evidence of robustness.
2. **The stress test is ours, and we developed against it.** Three rounds of fixes targeted the conditions it found weak, and the zoomed views (§6.3a) were chosen after seeing its weakest condition. Numbers on a test you have tuned against are optimistic.
3. **Unanticipated changes hurt.** The ablations (§6.4) are our only measurement of a change the model never trained on: without orientation training, parts turned 90° scored 0.563 (from 1.000); without camera effects, squeezed JPEGs scored 0.702 (from 0.990). If the private test contains a kind of change we did not simulate, a drop of that size on that portion is plausible.

The four conditions no version of our simulator ever made (tilted camera, motion blur, shadow, low resolution) score 0.99–1.00, which is encouraging but covers only four kinds of change, all of them designed by us.

7. Error analysis

Three rounds of stress → diagnose → fix. The first two found faults that validation (1.000) had hidden completely.

Round 1 — far-away parts read as contamination. v1 called *all 34* normal parts contamination at 0.65× scale, and none at 0.75× or above. The ablation located the cause: switching off *lighting* removed the fault (0.99). Lighting augmentation removes the colour cue, so the model learned contamination by **size** ("small spots"), and at a distance the three component dots are small spots. Fix: zoom range 0.85–1.12 → 0.6–1.2. Far away 0.71 → 1.00; very far 0.10 → 0.96.

Round 2 — dead pixels and sharpening. New never-simulated conditions broke every model: salt and pepper (v2: 0.047) and sharpening (0.523). They are the same small-spots failure in a new form: scattered dark pixels resemble specks, and sharpening halos turn a component dot into an outlined circle, which is a dent. Fix: both added to the camera family. 0.985 and 0.995.

Round 3 — what remains (docs/errors_full-v3.png : 45 errors in 6,000 stressed images with eight views; the confusion is almost entirely `normal → contamination` 16, `dent → normal` 8, `normal → missing_component` 5):

1. **Half-size parts** (very far, 0.783) — the residue of the round-1 fault; most of the errors. v2 scored 0.960 here, so v3's two extra camera effects traded some of it away. This is a single run per recipe, so part of the gap may be run-to-run variation.
2. **A glare square covering a component dot** → `missing_component` , at confidence ≈ 1.00 . Genuinely ambiguous: a component hidden by glare is indistinguishable from a missing one in the image. The right answer is REVIEW, and the model is too sure. This is our clearest calibration failure.
3. **Heavy blur, low resolution, dead pixels** → **a dent read as normal**, once the dent's outline is gone.
4. **Sharpening** → **a scratch read as normal** (2 cases).

Right for the right reasons. Grad-CAM heat-maps (docs/heatmaps_final.png) concentrate on the white dot (`missing_component`), the lines (`scratch`), the outlined circles (`dent`), the crossing (`misalignment`) and the

specks (`contamination`), and not on glare or background.

8. Final approach

1. `standardise()` every image to 128 × 128 RGB.
2. **Three final ResNet18s** — the v3 recipe with seeds 42, 1 and 2, each trained on all 950 images.
3. **Sixteen views per image** — rotations 0/90/180/270°, each also mirrored, at zoom 1.0 and 1.3.
4. Softmax at T = 1 (calibration only ever allowed to reduce confidence; §6.5), probabilities averaged over the three models.
5. One of the six classes per image, with its probability as the confidence, in the organisers' exact format (`sample_id,predicted_class,confidence` ; header verified against `sample_submission.csv`). `python predict.py <folder>` .

Unknown / uncertain handling. The CSV always names one of the six classes, because the DefectLens README defines exactly six; unlike the other two tracks, it does not mention an "unknown" class. Uncertainty is carried by the confidence. In the prototype, anything below **0.60** becomes **REVIEW**, which routes the part to a person instead of letting the machine pass or scrap it.

Working prototype (`app.py` + `static/index.html` , Flask, no external dependencies):

- verdict PASS / REJECT / REVIEW with confidence and all six probabilities;
- a Grad-CAM heat-map of where the model looked;
- **"stress it" buttons** — turn, relight, change background, glare, blur, grain, distance. They stack, so anyone can test the robustness claim live;
- batch inspection of a folder with a downloadable `submission.csv` ;
- accepts photos of any size.

9. Limitations

- **The stress conditions are our guesses.** They cover what the organisers named, but the private test's exact shifts may differ, and "defect presentation" (a scratch drawn differently, a new kind of dent) is the hardest to simulate: we can disturb conditions around a defect far more easily than we can re-draw the defect.
- **Very small parts** vary most between training runs (0.78–0.93 for one model, one view); the submitted ensemble reaches 0.990 there, partly by design choices made on the same stress set.
- **Over-confidence on genuine ambiguity** (glare over a component).
- **One run per recipe for the ablations.** Only the final recipe was repeated (three seeds, ±0.002 on the stress average). Differences of a few thousandths between the ablation models are within that variation; only the large effects in §6–7 should be read as real.
- **Stress-test leakage.** The disturbance functions used for scoring are the ones used in training, which is why each condition is labelled by kind and the never-simulated set is reported separately.
- **Synthetic data only.** Nothing here has been checked on real camera images.

10. Future improvements

- **Ensembles:** several seeds and a second architecture, averaged, to reduce variance and cover individual blind spots (v2 and v3 fail differently).
 - **Repeated runs** per recipe, to put error bars on every number in §6.
 - **Scale normalisation:** locate the part and crop to it before classifying, making distance irrelevant rather than learned.
 - **Synthesised new defect presentations** (lighter scratches, differently shaped dents) as a held-out "never simulated" set for the defects themselves.
 - **Ambiguity-aware training:** label glare-over-component cases as uncertain, so the model learns to say REVIEW there.
-

Reproducing

```
python3.14 -m venv .venv && .venv/bin/pip install -r requirements.txt
# ImageNet weights for ResNet18 -> models/resnet18-imagenet.pth
curl -L -o models/resnet18-imagenet.pth https://download.pytorch.org/models/resnet18-f37072fd.pth
.venv/bin/python train.py --name plain --plain          # the naive baseline
.venv/bin/python train.py --name full-v3                # the final recipe, train only
.venv/bin/python evaluate.py plain full-v3 --detail full-v3 # stress test, errors, calibration
for s in 42 1 2; do .venv/bin/python train.py --final --seed $s --name final$([ $s = 42 ] || echo $s)
.venv/bin/python predict.py <folder of test images>      # -> submission.csv
.venv/bin/python app.py                                  # the station, http://localhost:8050
```

Data goes in `data/DefectLens/{train,validation}/<class>/`. Seed 42 throughout; each training run takes about 2 minutes on an Apple M3 Pro.